Kinematics of ground robots

Juan Francisco Rascón Crespo

Saturday 7th March, 2026

Abstract

This paper presents a rigorous analytical development of forward and inverse kinematics for wheeled ground robots. Starting from the definition of an invariant base coordinate frame for each wheel, a system of matrix equations is formulated that relates the individual wheel velocities to the state of motion of the chassis. Since platforms with multiple wheels usually generate overdetermined systems, their optimal resolution is addressed using the least squares method and the Moore-Penrose pseudoinverse. The text places a fundamental emphasis on software implementation and numerical stability, justifying in detail the use of Singular Value Decomposition (SVD) to circumvent algebraic instabilities. Additionally, condition number analysis (κ) is introduced as a critical diagnostic tool to distinguish between the computational robustness of the algorithm and the physical feasibility of the mechanical design. Finally, this generalized theoretical framework is applied to the formulation and kinematic resolution of various standard topologies in mobile robotics, ranging from differential traction and Ackermann steering systems, to omnidirectional platforms with Mecanum wheels and *swerve drive* configurations.

1 Motion constraints imposed by wheels

In a ground robot, the wheels impose motion constraints:

- **Rolling constraint:** the wheel can only move in the rolling direction.
- **No lateral slip constraint (conventional wheel):** the contact velocity component between wheel and ground in the direction perpendicular to rolling is zero.
- **Omnidirectional/Swedish wheels:** this lateral constraint does not strictly apply, as the rollers allow passive lateral motion.

These constraints can be expressed mathematically as equations that relate the linear and angular velocity of the wheel to the robot velocity. These

equations are fundamental to the design of controllers and motion planners for ground robots, as they determine how the robot can move in its environment.

By stacking these constraints for each wheel, the forward and inverse kinematics equations can be derived for different configurations of ground robots, such as robots with differential wheels, robots with omnidirectional wheels, and robots with swerve drives, among others.

Figure 1 shows the 2D positioning of a wheel with respect to the robot frame, $\mathcal{R}$. The term d represents the distance from the robot frame to the wheel-ground contact point. The term α represents the angle that the $X_{\mathcal{R}}$ axis makes with the wheel position vector, $\boldsymbol{d}$, and because of how the wheel is positioned, it is also the angle that the $X_{\mathcal{R}}$ axis makes with the $Y_{\mathcal{O}}$ axis. The $X_{\mathcal{O}}$ axis is perpendicular to the $Y_{\mathcal{O}}$ axis and its orientation with respect to the $X_{\mathcal{R}}$ axis is $\alpha - \frac{\pi}{2}$.

$$\boldsymbol{d} = \begin{bmatrix} d\cos\alpha \\ d\sin\alpha \\ 0 \end{bmatrix} \tag{1}$$

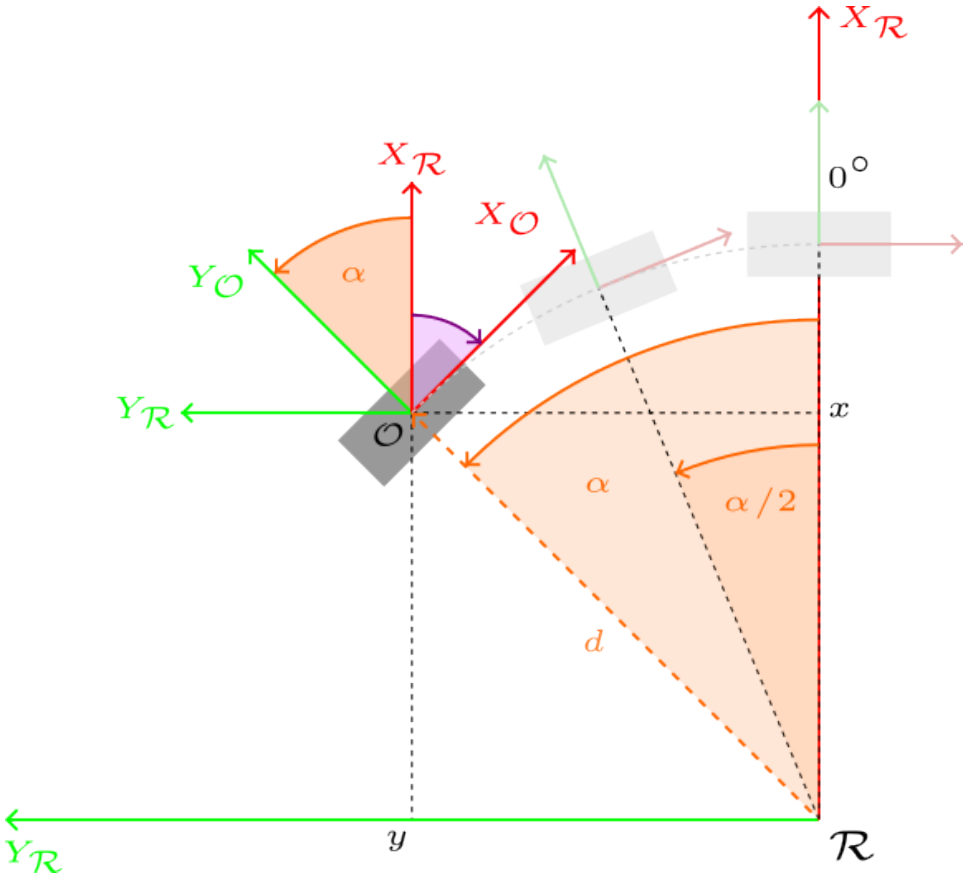

Figure 1: 2D positioning of a wheel with respect to the robot frame.

Next, an angle β is considered that allows the wheel to be oriented in the desired way, as shown in figure 2. The final orientation of the wheel is given by the angle ψ, formed between the $X_{\mathcal{R}}$ axis and the $X_{\mathcal{B}}$ axis.

$$\psi = \alpha + \beta - \frac{\pi}{2}$$

and, to simplify the notation, the symbol ϕ is introduced as:

$$\phi = \alpha + \beta, \quad \psi = \phi - \frac{\pi}{2}$$

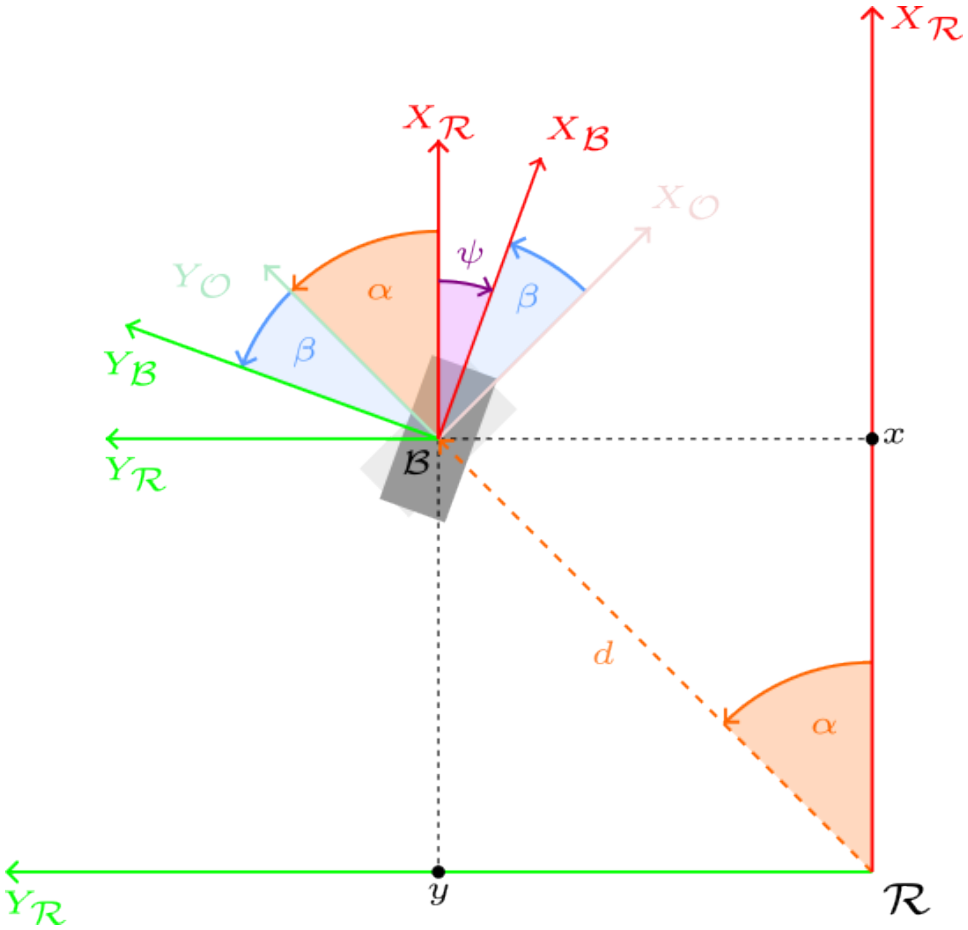

Figure 2: Wheel orientation using angle β.

The frame $\mathcal{B}$ is called the **wheel base frame**, and its position and orientation remain constant with respect to frame $\mathcal{R}$. In non-steerable wheels, the $X_{\mathcal{B}}$ axis determines the rolling direction of the wheel and the $Y_{\mathcal{B}}$ axis is the wheel rotation axis. In steerable wheels, the frame fixed to the wheel, $\mathcal{W}_i$, changes its orientation (angle θ) with respect to frame $\mathcal{B}$, and they coincide only when $\theta = 0$. See Figure 4.

The linear velocity of the base, expressed in the robot frame, is denoted as:

$$ {}^{\mathcal{R}}\boldsymbol{V}_r = \begin{bmatrix} {}^{\mathcal{R}}V_{r,x} \\ {}^{\mathcal{R}}V_{r,y} \\ 0 \end{bmatrix} \tag{2} $$

and is expressed in m/s.

The angular velocity of the base is denoted as:

$$\boldsymbol{\omega}_r = \begin{bmatrix} 0 \\ 0 \\ \omega_{r,z} \end{bmatrix} \tag{3}$$

and is expressed in rad/s.

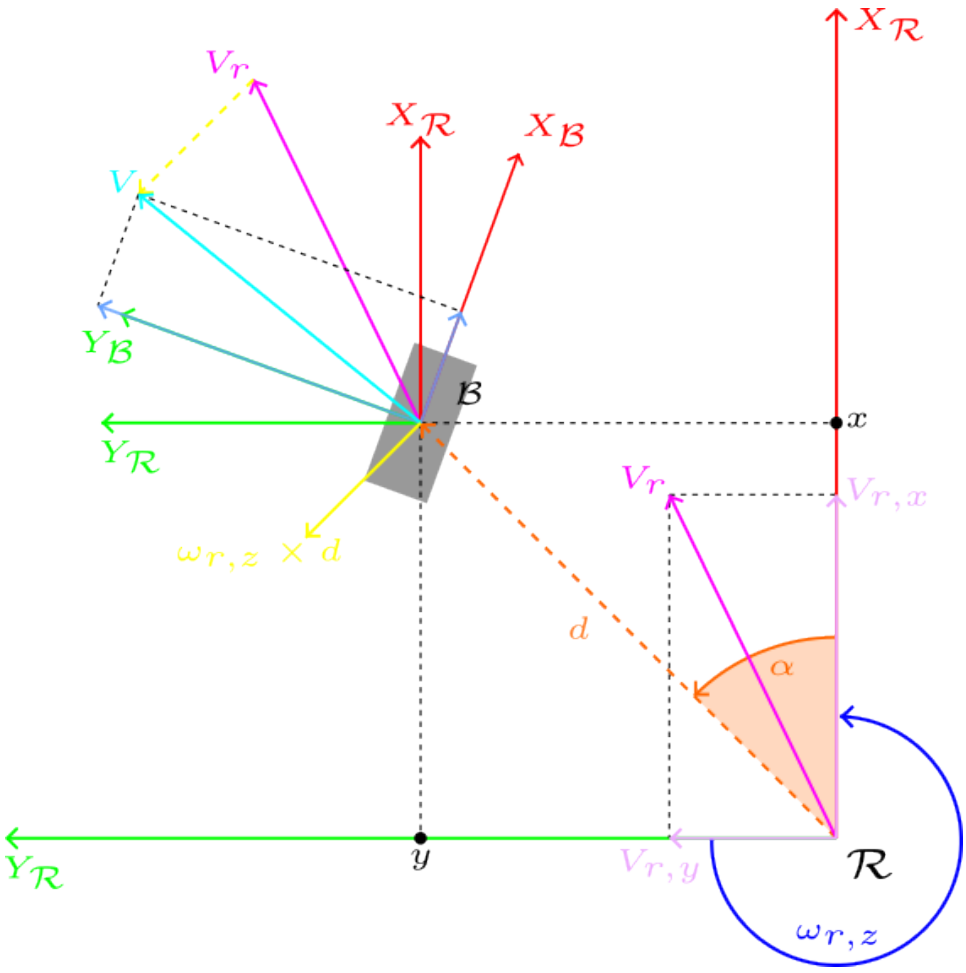

Figure 3: Relevant linear and angular velocities for the wheel.

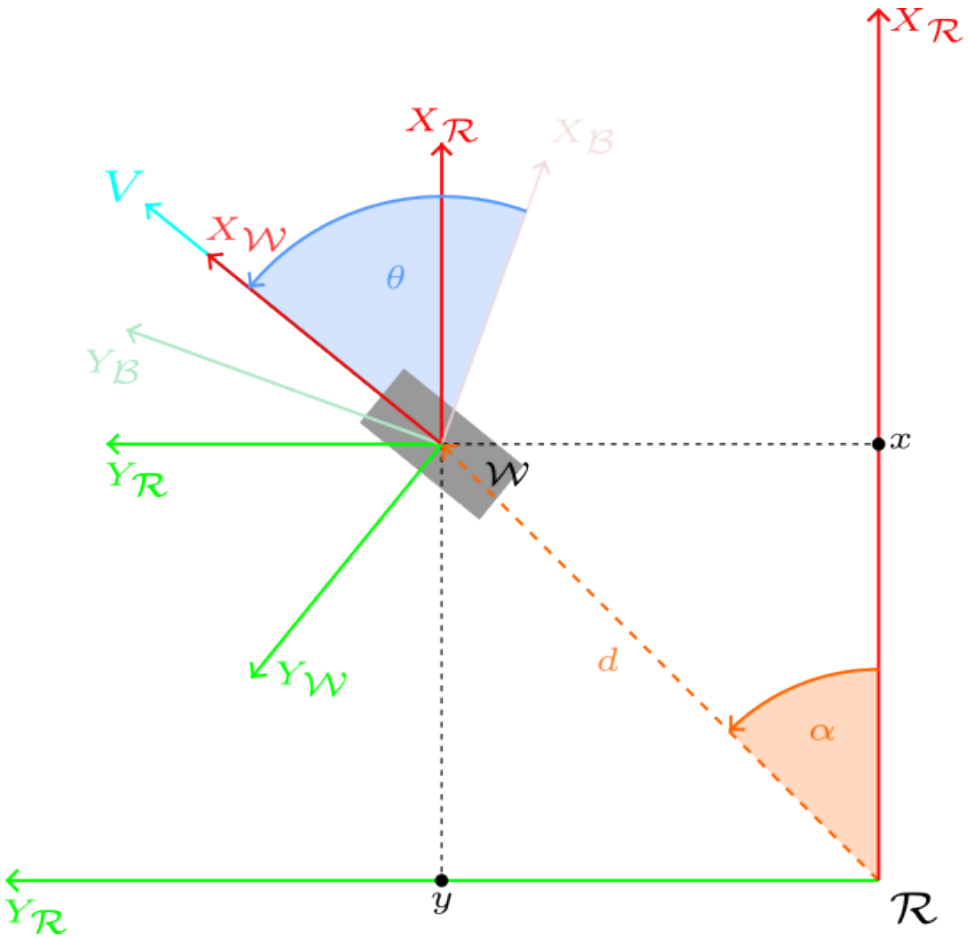

Figure 4: Conventional steerable wheel rotating an angle θ with respect to frame $\mathcal{B}$ to achieve the appropriate rolling direction.

The linear velocity of each wheel i, expressed in frame $\mathcal{R}$, is denoted as:

$$ {}^{\mathcal{R}}\boldsymbol{V}_i = {}^{\mathcal{R}}\boldsymbol{V}_r + \boldsymbol{\omega}_r \times {}^{\mathcal{R}}\boldsymbol{d}_i = {}^{\mathcal{R}}\boldsymbol{V}_r + [\boldsymbol{\omega}_r]_\times \, {}^{\mathcal{R}}\boldsymbol{d}_i \tag{4} $$

where ${}^{\mathcal{R}}\boldsymbol{d}_i$ is the position vector of wheel i with respect to frame $\mathcal{R}$, and $[\boldsymbol{\omega}_r]_\times$ is the antisymmetric operator that represents the cross product with $\boldsymbol{\omega}_r$.

The vector product between $\boldsymbol{\omega}_r$ and ${}^{\mathcal{R}}\boldsymbol{d}_i$ can be written as a matrix product using the antisymmetric operator $[\cdot]_\times$: $\boldsymbol{\omega}_r \times {}^{\mathcal{R}}\boldsymbol{d}_i = [\boldsymbol{\omega}_r]_\times \, {}^{\mathcal{R}}\boldsymbol{d}_i$, and $[\boldsymbol{\omega}_r]_\times$ is defined as:

$$ [\boldsymbol{\omega}_r]_\times \triangleq \begin{bmatrix} 0 & -\omega_{r,z} & \omega_{r,y} \\ \omega_{r,z} & 0 & -\omega_{r,x} \\ -\omega_{r,y} & \omega_{r,x} & 0 \end{bmatrix} \tag{5} $$

$$
\begin{aligned}
\boldsymbol{\omega}_r \times {}^{\mathcal{R}}\boldsymbol{d}_i &= [\boldsymbol{\omega}_r]_\times \, {}^{\mathcal{R}}\boldsymbol{d}_i \\
&= \begin{bmatrix} 0 & -\omega_{r,z} & 0 \\ \omega_{r,z} & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} d_i \cos\alpha_i \\ d_i \sin\alpha_i \\ 0 \end{bmatrix} \\
&= \begin{bmatrix} -\omega_{r,z} d_i \sin\alpha_i \\ \omega_{r,z} d_i \cos\alpha_i \\ 0 \end{bmatrix}
\end{aligned} \tag{6}
$$

$$
\begin{aligned}
{}^{\mathcal{R}}\boldsymbol{V}_i = {}^{\mathcal{R}}\boldsymbol{V}_r + \boldsymbol{\omega}_r \times {}^{\mathcal{R}}\boldsymbol{d}_i &= \begin{bmatrix} {}^{\mathcal{R}}V_{r,x} - \omega_{r,z} d_i \sin\alpha_i \\ {}^{\mathcal{R}}V_{r,y} + \omega_{r,z} d_i \cos\alpha_i \end{bmatrix} \\
&= \begin{bmatrix} 1 & 0 & -d_i \sin\alpha_i \\ 0 & 1 & d_i \cos\alpha_i \end{bmatrix} \begin{bmatrix} {}^{\mathcal{R}}V_{r,x} \\ {}^{\mathcal{R}}V_{r,y} \\ \omega_{r,z} \end{bmatrix}
\end{aligned} \tag{7}
$$

The vector ${}^{\mathcal{R}}\boldsymbol{V}_i$ represents the linear velocity of wheel i in frame $\mathcal{R}$. To apply motion constraints to each wheel, its velocity must be expressed in its base frame, $\mathcal{B}_i$, ${}^{\mathcal{B}_i}V_i$.

$$
{}^{\mathcal{B}_i}\boldsymbol{V}_i = {}^{\mathcal{B}_i}R(\psi_i)_3 \, {}^{\mathcal{R}}\boldsymbol{V}_i \tag{8}
$$

where the rotation matrix ${}^{\mathcal{B}_i}R(\psi_i)_3$ represents the orientation of frame $\mathcal{R}$ in frame $\mathcal{B}_i$, and is used to convert vectors expressed in frame $\mathcal{R}$ to the base frame of wheel i, $\mathcal{B}_i$.

The matrix that is known a priori is:

$$
{}^{\mathcal{R}}R(\psi_i)_{\mathcal{B}_i} = \begin{bmatrix} \cos\psi_i & -\sin\psi_i \\ \sin\psi_i & \cos\psi_i \end{bmatrix} \tag{9}
$$

with $\psi_i = \phi_i - \frac{\pi}{2} = \alpha_i + \beta_i - \frac{\pi}{2}$, which represents the orientation of frame $\mathcal{B}_i$ with respect to frame $\mathcal{R}$.

Since rotation matrices are orthogonal, their inverse is equal to their transpose, so:

$$
{}^{\mathcal{B}_i}R(\psi_i)_{\mathcal{R}} = \left({}^{\mathcal{R}}R(\psi_i)_{\mathcal{B}_i}\right)^{-1} = \left({}^{\mathcal{R}}R(\psi_i)_{\mathcal{B}_i}\right)^{\mathsf{T}} = \begin{bmatrix} \cos\psi_i & \sin\psi_i \\ -\sin\psi_i & \cos\psi_i \end{bmatrix} \tag{10}
$$

The following trigonometric identities are applied to ${}^{\mathcal{B}_i}R(\psi_i)_{\mathcal{R}}$:

$$\cos\psi_i = \cos\left(\phi_i - \frac{\pi}{2}\right) = \quad \sin\phi_i$$
$$\sin\psi_i = \sin\left(\phi_i - \frac{\pi}{2}\right) = -\cos\phi_i$$

finally remaining:

$$^{\mathcal{B}_i}R(\phi_i)_{\mathcal{R}} = \begin{bmatrix} \sin\phi_i & -\cos\phi_i \\ \cos\phi_i & \sin\phi_i \end{bmatrix} \tag{11}$$

$$\begin{aligned} ^{\mathcal{B}_i}\boldsymbol{V}_i &= {}^{\mathcal{B}_i}R(\phi_i)_{\mathcal{R}}{}^{\mathcal{R}}\boldsymbol{V}_i \\ &= \begin{bmatrix} \sin\phi_i & -\cos\phi_i \\ \cos\phi_i & \sin\phi_i \end{bmatrix} \begin{bmatrix} 1 & 0 & -d_i\sin\alpha_i \\ 0 & 1 & d_i\cos\alpha_i \end{bmatrix} \begin{bmatrix} {}^{\mathcal{R}}V_{r,x} \\ {}^{\mathcal{R}}V_{r,y} \\ \omega_{r,z} \end{bmatrix} \end{aligned} \tag{12}$$

The velocity vector $^{\mathcal{B}_i}\boldsymbol{V}_i$ can be expressed, in general, as:

$$^{\mathcal{B}_i}\boldsymbol{V}_i = \begin{bmatrix} v_{i\parallel} \\ v_{i\perp} \end{bmatrix} \tag{13}$$

$v_{i\parallel}$ being the velocity component in the rolling direction, $X_{\mathcal{W}_i}$, and $v_{i\perp}$ being the velocity component perpendicular to the rolling direction (in other words, the lateral velocity of the wheel, parallel to the axis $Y_{\mathcal{W}_i}$).

$$\begin{bmatrix} v_{i\parallel} \\ v_{i\perp} \end{bmatrix} = \begin{bmatrix} \sin\phi_i & -\cos\phi_i \\ \cos\phi_i & \sin\phi_i \end{bmatrix} \begin{bmatrix} 1 & 0 & -d_i\sin\alpha_i \\ 0 & 1 & d_i\cos\alpha_i \end{bmatrix} \begin{bmatrix} {}^{\mathcal{R}}V_{r,x} \\ {}^{\mathcal{R}}V_{r,y} \\ \omega_{r,z} \end{bmatrix} \tag{14}$$

$$\begin{bmatrix} v_{i\parallel} \\ v_{i\perp} \end{bmatrix} = \begin{bmatrix} \sin\phi_i & -\cos\phi_i & -d_i(\ \ \sin\alpha_i\sin\phi_i + \cos\alpha_i\cos\phi_i) \\ \cos\phi_i & \sin\phi_i & d_i(-\sin\alpha_i\cos\phi_i + \cos\alpha_i\sin\phi_i) \end{bmatrix} \begin{bmatrix} {}^{R}V_{r,x} \\ {}^{R}V_{r,y} \\ \omega_{r,z} \end{bmatrix}$$

The following trigonometric identities are then applied:

$$\begin{aligned} \sin\alpha_i\sin\phi_i + \cos\alpha_i\cos\phi_i &= \cos(\phi_i - \alpha_i) = \cos((\alpha_i + \beta_i) - \alpha_i) = \cos\beta_i \\ -\sin\alpha_i\cos\phi_i + \cos\alpha_i\sin\phi_i &= \sin(\phi_i - \alpha_i) = \sin((\alpha_i + \beta_i) - \alpha_i) = \sin\beta_i \end{aligned}$$

Finally resulting:

$$\begin{bmatrix} v_{i\parallel} \\ v_{i\perp} \end{bmatrix} = \begin{bmatrix} \sin\phi_i & -\cos\phi_i & -d\cos\beta_i \\ \cos\phi_i & \sin\phi_i & d\sin\beta_i \end{bmatrix} \begin{bmatrix} {}^{\mathcal{R}}V_{r,x} \\ {}^{\mathcal{R}}V_{r,y} \\ \omega_{r,z} \end{bmatrix} \tag{15}$$

For each wheel i, a system of 2 linear equations with 3 unknowns is obtained (${}^{\mathcal{R}}V_{r,x}$, ${}^{\mathcal{R}}V_{r,y}$ and $\omega_{r,z}$).

In general, for N wheels one obtains $2N$ linear equations and 3 unknowns. Therefore, for $N \geq 2$ the system is overdetermined.

Depending on whether the wheels used on a platform are non-steerable, steerable, mecanum or omnidirectional, different constraints are imposed on $v_{i\parallel}$ and $v_{i\perp}$, leading to different ways of solving forward and inverse kinematics.

In the study of each specific platform, the specific constraints of each type of wheel will be applied to the previous equations to obtain the corresponding forward and inverse kinematics formulas.

2 General considerations on forward and inverse kinematics

In general, the forward and inverse kinematics of a terrestrial robot are defined by a matrix expression of the form:

$$A\mathbf{x} = \mathbf{b} \tag{16}$$

where:

$$A \in \mathbb{R}^{2N\times 3}, \quad \mathbf{x} \in \mathbb{R}^3, \quad \mathbf{b} \in \mathbb{R}^{2N}$$

$$A = \begin{bmatrix} \sin\phi_1 & -\cos\phi_1 & -d_1\cos\beta_1 \\ \cos\phi_1 & \sin\phi_1 & d_1\sin\beta_1 \\ \vdots & \vdots & \vdots \\ \sin\phi_N & -\cos\phi_N & -d_N\cos\beta_N \\ \cos\phi_N & \sin\phi_N & d_N\sin\beta_N \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} {}^{\mathcal{R}}V_{r,x} \\ {}^{\mathcal{R}}V_{r,y} \\ \omega_{r,z} \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} v_{1\parallel} \\ v_{1\perp} \\ \vdots \\ v_{N\parallel} \\ v_{N\perp} \end{bmatrix}$$

Given that for any robot with $N \geq 2$ wheels, $2N$ equations and only 3 unknowns are obtained, we are faced with an overdetermined linear system.

In an ideal purely mathematical scenario, it would be enough to take any subset of 3 linearly independent equations to solve the system and find the

chassis velocity, $\mathbf{x}$. However, in the actual implementation of a ground robot, the equations never fit perfectly due to imperfections such as:

- Noise and quantization in the reading of sensors (encoders).
- Small slipping of the wheels on the ground.
- Slight deviations in the geometric calibration of the base.

Therefore, the system is not solved by searching for an exact algebraic solution, but rather by searching for the estimate $\hat{\mathbf{x}}$ that minimizes the global quadratic error of all the wheels simultaneously:

$$\hat{\mathbf{x}} = \arg\min_{\mathbf{x}} \|A\mathbf{x} - \mathbf{b}\|^2 \tag{17}$$

The solution to this least squares problem is given by the expression known as Normal Equations, which is obtained from a process of searching for the minimum of the error function (differentiating and equating to zero):

$$A^\mathsf{T} A\mathbf{x} = A^\mathsf{T}\mathbf{b} \tag{18}$$

Therefore, the optimal solution is obtained by:

$$\hat{\mathbf{x}} = (A^\mathsf{T} A)^{-1} A^\mathsf{T}\mathbf{b} = A^{+}\mathbf{b} \tag{19}$$

where A^+ is called the Moore-Penrose pseudoinverse of the A matrix.

From the point of view of computational numerical calculation, directly inverting the matrix $(A^\mathsf{T} A)$ can be unstable if the original matrix A is poorly conditioned. A matrix is considered ill-conditioned when small variations or noise in the input data (the $\mathbf{b}$ vector) are dramatically amplified, causing erratic calculations in the resulting $\hat{\mathbf{x}}$ solution.

The poor numerical behavior that can arise when using the normal equations formula is because the term $(A^\mathsf{T} A)^{-1} A^\mathsf{T}$ squares the singular values of A. If A has a small singular value, close to zero, squaring it makes it even smaller. When trying to invert that value, the computer divides by a number almost zero, which causes the results to shoot up to infinity and all precision in the solution is lost.

Although it is not strictly necessary for the kinematic derivation, for completeness it is useful to assess the geometric robustness of matrix A using the ratio between its maximum and minimum singular values ($\kappa = \frac{\sigma_{\max}}{\sigma_{\min}}$), called the **condition number**, $\kappa(A)$. As a reference:

- $\kappa = 1$: A is the perfect matrix (example: the Identity matrix or a pure rotation matrix). There is no loss of precision.
- $1 < \kappa < 10^2$: A is well conditioned. The system is robust; Sensor errors will not be significantly amplified.
- $10^3 < \kappa < 10^6$: A is moderately ill-conditioned. Caution is required, since simple methods such as inverting $(A^\mathsf{T}A)$ may start to produce inaccurate results.
- $\kappa \approx 10^{12}$ a 10^{15}: A is poorly conditioned. The matrix is almost singular (its rows/columns are almost dependent).
- $\kappa > 10^{16}$: A is singular and unusable. For a computer working with double precision (64 bits), this means that the matrix cannot be reliably inverted.

To avoid these numerical stability problems that could arise, in practice the pseudoinverse A^+ is obtained by applying the Singular Value Decomposition (SVD) to the original A matrix. SVD is the most robust and secure numerical method, since it completely avoids the product $(A^\mathsf{T}A)$ and is capable of handling quasi-singular matrices (situations where the geometric equations of the robot are so similar or redundant that the system is on the verge of not being able to be inverted) without the resulting calculations shooting to infinity. This method works even if the A matrix does not have full rank (if some of the equations are directly contradictory).

However, it is essential to make a critical distinction between the mathematical robustness of the software and the physical viability of the robot. If evaluating the constant geometric matrix A during the design phase results in a high condition number (for example, $\kappa > 10^3$, and critically if $\kappa > 10^6$), **it will happen that, although the SVD algorithm avoids program collapse by returning a stable mathematical solution (by truncating near-zero values), it will actually be masking a serious defect in the mechanical design.** Physically, this means that the constructive arrangement of the wheels (their d_i distances or α_i, β_i mounting angles) generates degenerate kinematics, where the chassis loses the ability to move independently in all its degrees of freedom, or where the rolling directions come into direct conflict. In this scenario, even if the software delivers an apparently valid numerical result, the physical robot will suffer uncontrollable drifts, mechanical stress, and erratic motion at the slightest noise in the encoders. Therefore, SVD should not be used as a pretext to ignore bad geometry; if the initial condition number is excessively high, the unavoidable solution is to rethink the base location and orientation of the wheels in the CAD design until a well-conditioned system is achieved (ideally $\kappa \leq 100$).

The SVD decomposes the matrix into $A = U\Sigma V^\mathsf{T}$, allowing the pseudoinverse to be constructed directly as $A^+ = V\Sigma^+ U^\mathsf{T}$.

Derivation:

If $A = U\Sigma V^\mathsf{T}$, the transpose property is applied for a product of matrices: $(XYZ)^\mathsf{T} = Z^\mathsf{T} Y^\mathsf{T} X^\mathsf{T}$ (the order is reversed).

$$A^\mathsf{T} = (V^\mathsf{T})^\mathsf{T} \Sigma^\mathsf{T} U^\mathsf{T} \tag{20}$$

Transposing twice returns the original matrix ($(V^\mathsf{T})^\mathsf{T} = V$), and since Σ is a diagonal matrix that does not change when transposed ($\Sigma^\mathsf{T} = \Sigma$)[1], we obtain:

$$A^\mathsf{T} = V\Sigma U^\mathsf{T} \tag{21}$$

Substituting into product $A^\mathsf{T} A$:

$$A^\mathsf{T} A = (V\Sigma U^\mathsf{T})(U\Sigma V^\mathsf{T}) \tag{22}$$

Due to the associative property of matrices, the internal parentheses can be omitted:

$$A^\mathsf{T} A = V\Sigma (U^\mathsf{T} U)\Sigma V^\mathsf{T} \tag{23}$$

The U matrix is orthogonal, meaning that its transpose multiplied by itself results in the Identity matrix ($U^\mathsf{T} U = I$):

$$A^\mathsf{T} A = V\Sigma I \Sigma V^\mathsf{T} \tag{24}$$

Multiplying by the Identity matrix does not alter the result, and Σ multiplied by itself is Σ^2 (which is equivalent to squaring the values of its diagonal):

$$A^\mathsf{T} A = V\Sigma^2 V^\mathsf{T} \tag{25}$$

[1]For the equality $\Sigma^\mathsf{T} = \Sigma$ to be strictly true, Σ has to be a square matrix. In overdetermined systems, where A is a rectangular matrix of $M \times N$, the original Σ matrix is rectangular of $M \times N$ and Σ^T would be of $N \times M$, so the equality would not hold. However, in practice, to solve these systems, Reduced SVD (or *Thin SVD*) is usually used, which cuts the rows of zeros from the Σ matrix, converting it into a square matrix of size $N \times N$. In this way, it is true that $\Sigma^\mathsf{T} = \Sigma$ in the Reduced SVD, which allows the $A^+ = V\Sigma^+ U^\mathsf{T}$ formula to be developed without dimensional conflicts. In the Eigen C++ library, the option combination `ComputeThinU | ComputeThinV` performs exactly this calculation.

The inverse is then applied to the entire expression using the inverse property for a product: $(XYZ)^{-1} = Z^{-1}Y^{-1}X^{-1}$.

$$(A^\mathsf{T}A)^{-1} = (V^\mathsf{T})^{-1}(\Sigma^2)^{-1}V^{-1} \tag{26}$$

Since V is also an orthogonal matrix, its inverse is equal to its transpose, $V^{-1} = V^\mathsf{T}$, and therefore $(V^\mathsf{T})^{-1} = V$. The inverse of a diagonal matrix Σ^2 is denoted simply as Σ^{-2} (which consists of inverting the squared values of its diagonal). Substituting these properties, we obtain:

$$(A^\mathsf{T}A)^{-1} = V\Sigma^{-2}V^\mathsf{T} \tag{27}$$

It is now multiplied by A^T on the right to complete the expression of the Normal Equations:

$$(A^\mathsf{T}A)^{-1}A^\mathsf{T} = (V\Sigma^{-2}V^\mathsf{T})(V\Sigma U^\mathsf{T}) \tag{28}$$

The internal parentheses are removed again for the associative property:

$$(A^\mathsf{T}A)^{-1}A^\mathsf{T} = V\Sigma^{-2}(V^\mathsf{T}V)\Sigma U^\mathsf{T} \tag{29}$$

Again, since V is orthogonal, $V^\mathsf{T}V = I$. The central term disappears:

$$(A^\mathsf{T}A)^{-1}A^\mathsf{T} = V\Sigma^{-2}\Sigma U^\mathsf{T} \tag{30}$$

At this point we have Σ^{-2} multiplied by Σ. Because both are diagonal matrices, the operation is equivalent to multiplying each element of the diagonal of Σ^{-2} by the corresponding element of the diagonal of Σ. So each element of the resulting diagonal is the original value of Σ raised to $-2 + 1 = -1$ (i.e., the inverse of the original values of Σ). This resulting diagonal matrix is denoted as Σ^+:

- If A **does not have** full rank, Σ^+ is defined as the diagonal matrix containing the inverse of the non-null singular values of A, and zeros in the positions corresponding to the null singular values.
- If A **has** full rank, then Σ^+ is actually Σ^{-1} (a proper inverse), which is the diagonal matrix containing the inverse of all singular values of A.

Finally, substituting Σ^+, we arrive at the expression:

$$(A^\mathsf{T}A)^{-1}A^\mathsf{T} = V\Sigma^+U^\mathsf{T} \tag{31}$$

As mentioned above, the computational problem arises when the computer evaluates the expression $(A^{\mathsf{T}}A)^{-1}$ directly, since it must square singular values that could be very close to zero. Trying to invert those squared near-zero values results in numerical overflows. Using the final formula $V\Sigma^{+}U^{\mathsf{T}}$ directly, as provided by the SVD, the algorithm only inverts the original values (Σ^{+}), keeping the system stable at all times.

3 Kinematics of a robot with a three swerve drive base

The base with three active steerable wheels (three swerve drive) is an omnidirectional locomotion system. It allows a ground robot to move in any direction without first having to change the orientation of the chassis. Each wheel is oriented independently, so the robot gains maneuverability and flexibility.

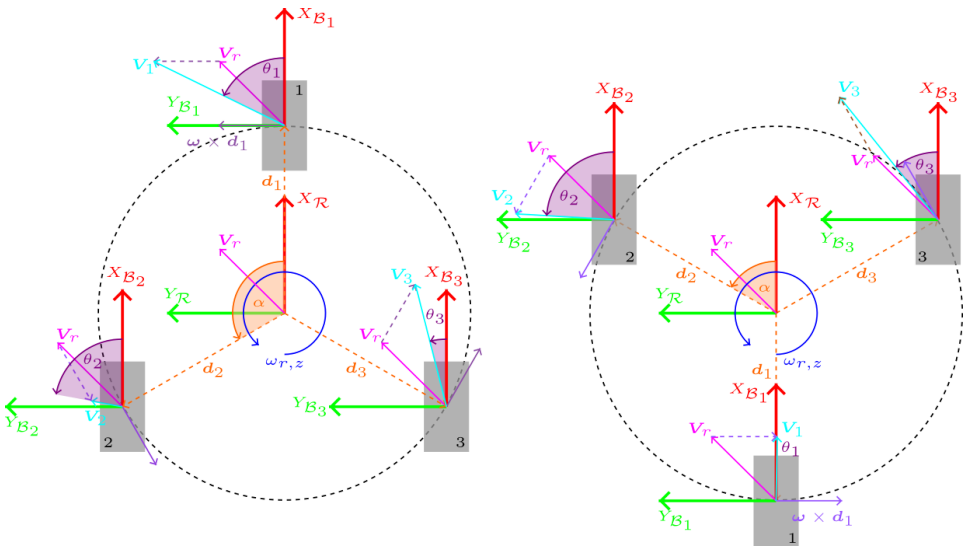

Figure 5: Common wheel configurations in a three swerve drive system (left: inverted Y configuration, right: Y configuration)

All wheels have the same radius r and are located at the same distance d from the center of the base.

Furthermore, in this particular case, we impose:

$$\psi_i = \phi_i - \frac{\pi}{2} = 0 \quad \Longrightarrow \quad \phi_i = \alpha_i + \beta_i = \frac{\pi}{2}, \quad \beta_i = \frac{\pi}{2} - \alpha_i$$

and therefore:

$$^{\mathcal{B}_i}R\left(\phi_i = \frac{\pi}{2}\right)_{\mathcal{R}} = \begin{bmatrix} \sin\frac{\pi}{2} & -\cos\frac{\pi}{2} \\ \cos\frac{\pi}{2} & \sin\frac{\pi}{2} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

For the inverted-Y configuration:

- $\alpha \in \left[\frac{\pi}{2}, \pi\right)$ and $d > 0$
- Front wheel ($i = 1$):

$$\alpha_1 = 0, \quad \boldsymbol{d}_1 = (d, 0), \quad \beta_1 = \frac{\pi}{2}$$

- Left wheel ($i = 2$):

$$\alpha_2 = \alpha, \quad \boldsymbol{d}_2 = (d\cos\alpha,\ d\sin\alpha), \quad \beta_2 = \frac{\pi}{2} - \alpha$$

- Right wheel ($i = 3$):

$$\alpha_3 = -\alpha, \quad \boldsymbol{d}_3 = (d\cos\alpha,\ -d\sin\alpha), \quad \beta_3 = \frac{\pi}{2} + \alpha$$

For Y configuration:

- $\alpha \in \left(0, \frac{\pi}{2}\right]$ and $d > 0$
- Rear wheel ($i = 1$):

$$\alpha_1 = \pi, \quad \boldsymbol{d}_1 = (-d, 0), \quad \beta_1 = -\frac{\pi}{2}$$

- Left wheel ($i = 2$):

$$\alpha_2 = \alpha, \quad \boldsymbol{d}_2 = (d\cos\alpha,\ d\sin\alpha), \quad \beta_2 = \frac{\pi}{2} - \alpha$$

- Right wheel ($i = 3$):

$$\alpha_3 = -\alpha, \quad \boldsymbol{d}_3 = (d\cos\alpha,\ -d\sin\alpha), \quad \beta_3 = \frac{\pi}{2} + \alpha$$

3.1 Inverse kinematics of the three swerve

Knowing the chassis velocity vector $^{\mathcal{R}}\boldsymbol{V}_r$, the velocity of each wheel i in its base frame $\mathcal{B}_i$, $^{\mathcal{B}_i}\boldsymbol{V}_i = [v_{i\parallel}, v_{i\perp}]^\mathsf{T}$, is obtained by applying the coordinate transformation of equation 15.

Since each wheel is steerable, the orientation angle θ_i that each wheel must adopt can be calculated, with respect to the base frame $\mathcal{B}_i$, so that its rolling direction is aligned with the velocity vector ${}^{\mathcal{B}_i}\boldsymbol{V}_i$.

$$\theta_i = \arctan 2(v_{i\perp}, v_{i\parallel}) \tag{32}$$

The angular velocity of the wheel is:

$$\|{}^{\mathcal{B}_i}\boldsymbol{V}_i\| = \sqrt{v_{i\parallel}^2 + v_{i\perp}^2} = r\omega_i \tag{33}$$

$$\omega_i = \frac{\|{}^{\mathcal{B}_i}\boldsymbol{V}_i\|}{r} = \frac{\sqrt{v_{i\parallel}^2 + v_{i\perp}^2}}{r} \tag{34}$$

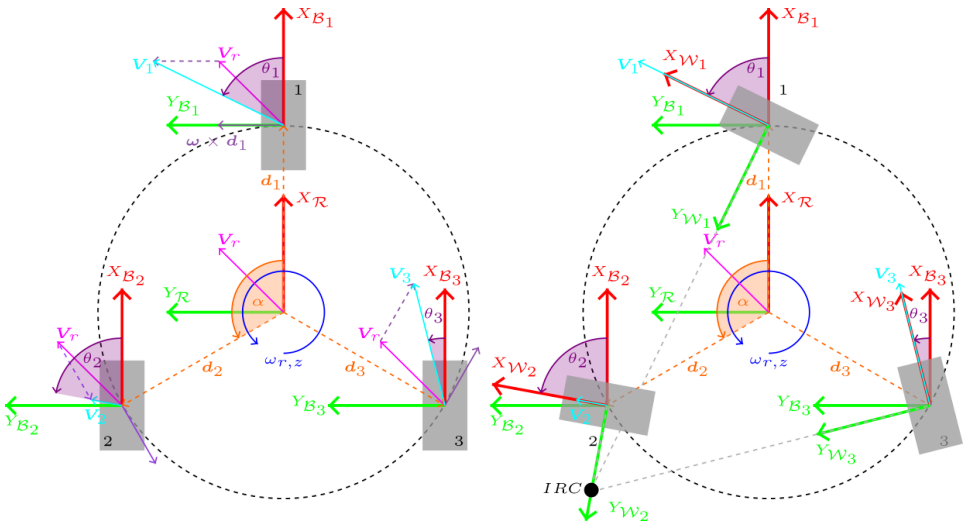

Figure 6: Inverse kinematics of a robot with inverted-Y configuration

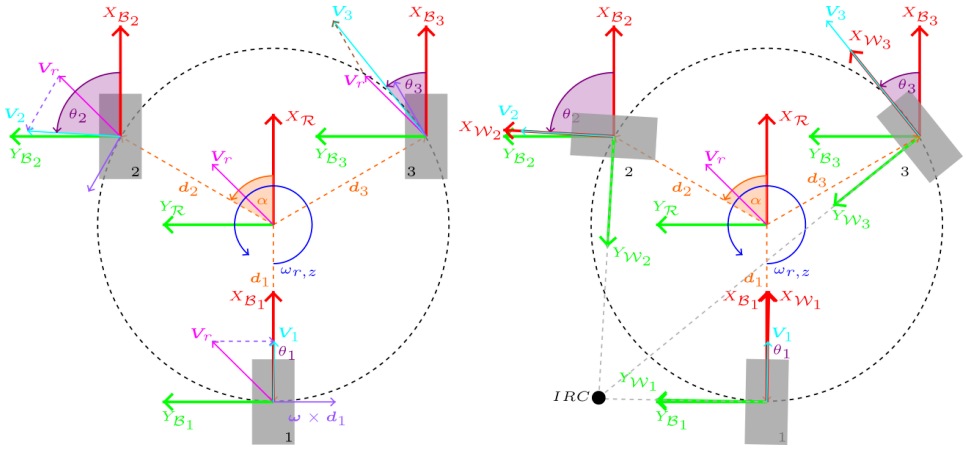

Figure 7: Inverse kinematics of a robot with Y configuration

3.2 Forward kinematics of the three swerve

Knowing the orientation angle θ_i and the angular velocity ω_i of each wheel i, the velocity vector of each wheel can be calculated in its base frame $\mathcal{B}_i$, ${}^{\mathcal{B}_i}\boldsymbol{V}_i = [v_{i\parallel}, v_{i\perp}]^\mathsf{T}$. Once all ${}^{\mathcal{B}_i}\boldsymbol{V}_i$ vectors are known, the vector $\mathbf{b} = [v_{1\parallel}, v_{1\perp}, \ldots, v_{N\parallel}, v_{N\perp}]^\mathsf{T}$ is constructed and the linear system $A\mathbf{x} = \mathbf{b}$ is solved to obtain the chassis velocity $\mathbf{x} = [{}^{\mathcal{R}}V_{r,x}, {}^{\mathcal{R}}V_{r,y}, \omega_{r,z}]^\mathsf{T}$.

From equation 33, the velocity components $v_{i\parallel}$ and $v_{i\perp}$ are obtained:

$$\begin{aligned} v_{i\parallel} &= \|{}^{\mathcal{B}_i}\boldsymbol{V}_i\| \cos\theta_i = r\omega_i \cos\theta_i \\ v_{i\perp} &= \|{}^{\mathcal{B}_i}\boldsymbol{V}_i\| \sin\theta_i = r\omega_i \sin\theta_i \end{aligned} \tag{35}$$

Below are the A matrices for the inverted-Y configuration and the Y configuration.

$$A_{Y-inv} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & d \\ 0 & -1 & -d\sin\alpha \\ 1 & 0 & d\cos\alpha \\ 0 & -1 & d\sin\alpha \\ 1 & 0 & d\cos\alpha \end{bmatrix}, \qquad A_Y = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & -d \\ 0 & -1 & -d\sin\alpha \\ 1 & 0 & d\cos\alpha \\ 0 & -1 & d\sin\alpha \\ 1 & 0 & d\cos\alpha \end{bmatrix}$$

The vector $\mathbf{b}$ is constructed from the velocities $v_{i\parallel}$ and $v_{i\perp}$ of each wheel:

$$\mathbf{b} = \begin{bmatrix} v_{1\parallel} \\ v_{1\perp} \\ v_{2\parallel} \\ v_{2\perp} \\ v_{3\parallel} \\ v_{3\perp} \end{bmatrix} \tag{36}$$

The $\mathbf{x}$ vector is calculated by applying the pseudoinverse of A, A^+, to the $\mathbf{b}$ vector as explained in Section 2.